

Phase 2 Report — ICSNADGym Environment & Sanity Checks

MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection

Generated 2026-07-08 · Builds on Phase 1's harmonized Parquet corpus (61 files, 60.1M rows) · Pipeline:

```
src/icsnad_gym/{env,reward,quarantine,sanity_check}.py
```

Why this report exists. Per the approved plan, no RL training result may be trusted until the environment passes API compliance and its reward function produces a sane policy ordering. This report documents the environment design, the checks run, and — importantly — a real implementation bug the sanity check caught before it could silently corrupt downstream training results.

1. Executive Summary

55

observation dims (49
numeric + 6 protocol one-
hot)

3

actions: ALLOW / ALERT
/ BLOCK

35

train-eligible files

PASS

check_env + sanity floor

Built `ICSNADGym`, a Gymnasium-compatible environment over real ICS-NAD flow telemetry; verified it against `gymnasium.utils.env_checker.check_env`; ran the five mandatory baseline policies (random, always-ALLOW, always-ALERT, always-BLOCK, oracle) and confirmed the reward function produces the expected dominance ordering. One real bug was found and fixed during this process — documented in Section 4.

2. Environment Design

2.1 Episodes

An episode is a contiguous, time-ordered window of `episode_length` flows (default 4096) sampled from a single capture file — never a synthetic cross-file mix, since different files are temporally and physically unrelated capture sessions. Window sampling is biased 70/30: with 70% probability the window is centered on a randomly chosen attack row (so attack exposure stays frequent despite real-world class imbalance), otherwise a uniformly random window is drawn (which may be pure BENIGN or incidentally include an attack).

2.2 Observation space

`Box`, shape (55,): 49 numeric flow features (z-score normalized using the Phase 1 per-brand scalers, train-split-fit only) plus a 6-category one-hot for `protocol` (IPv4-TCP, IPv4-UDP, IPv4-ICMP, ARP, IPv6,

OTHER). Identity fields — source/destination IP, MAC, absolute timestamps — are deliberately excluded so the policy can't shortcut by memorizing fixed addresses instead of learning traffic-shape patterns.

2.3 Action space & reward

`Discrete(3)` : `ALLOW` / `ALERT` (flag only) / `BLOCK` (alert + quarantine the flow's source for 50 subsequent flows). Base reward table, asymmetric by design (missing a real attack is the worst outcome):

Ground truth	Action	Reward
Benign	ALLOW	+1
Benign	ALERT	-5
Benign	BLOCK	-10
Attack	ALLOW	-20
Attack	ALERT	+10
Attack	BLOCK (first detection)	+15

Plus a detection-latency bonus ($5 / (1 + \text{flows_since_onset})$) on the first correct ALERT/BLOCK after a fresh attack onset, and a quarantine ongoing effect: +2/step while correctly holding a true attacker, -3/step while wrongly holding a benign source that got blocked — this is what makes BLOCK's statefulness matter beyond a single-step decision.

3. Verification

3.1 API compliance

`gymnasium.utils.env_checker.check_env` : **PASS**. (Two benign warnings about $\pm$ infinity observation bounds — expected and accepted, since z-scored continuous features are technically unbounded; this is standard practice for normalized RL observation spaces.)

3.2 Sanity-floor policy ordering

20 episodes per policy, episode_length=1024, train split, seed=0:

Policy	Mean reward	Std
oracle	4,356.15	2,573.58
always_ALERT	4,183.75	4,100.69
always_BLOCK	699.85	3,674.21
random	-851.40	1,455.22
always_ALLOW	-9,012.95	6,570.12

PASS — oracle strictly dominates (+172.4 over the best non-oracle policy, `always_ALERT`), and the full ordering (oracle > `always_ALERT` > `always_BLOCK` > random > `always_ALLOW`) matches expectations: `always-ALLOW` is clearly worst (pays -20 on every missed attack, and attack-heavy episodes are common by design), `always-ALERT` does surprisingly well because it never triggers the costlier `BLOCK-on-benign` penalty or false-quarantine cost, and oracle wins by acting correctly on both classes.

The oracle/`always_ALERT` margin is real but not huge — a legitimate consequence of `ALERT` being a structurally "safe" action in this reward design (no downside for over-alerting), not a flaw. The ordering itself, not the margin size, is what the sanity floor exists to verify.

4. Bug Found & Fixed During This Phase

Off-by-one in `info` dict caused the oracle policy to initially *fail* the sanity check. The first implementation of `step()` returned `info["is_attack"]` describing the row that was just scored, not the row matching the newly returned observation (i.e., the row the agent is about to act on next). Phase 1 had already established that attack/benign labels genuinely flicker row-by-row during floods (background traffic interleaves with attack flows at the flow-record level) — so this off-by-one meant a policy reading `info` to decide its next action was reacting to *stale* ground truth almost every step.

Effect before the fix: oracle scored 33.55 — *worse than* `always_BLOCK` (699.85) and far below `always_ALERT` (4,183.75), failing the "oracle must dominate" assertion outright. This is exactly the failure mode the sanity-floor check exists to catch: a subtle bug that would otherwise have silently produced misleading RL training curves with no obvious symptom until much later. Fixed by moving the `info` construction in `step()` to after the pointer advance, so it describes the upcoming row consistently with both `reset()` and `step()`.

5. Files Created

Module	Purpose
<code>env.py</code>	<code>ICSNADGym</code> : episode sampling, observation/action spaces, step/reset logic

<code>reward.py</code>	Base reward table, detection-latency bonus, quarantine ongoing reward
<code>quarantine.py</code>	Per-source quarantine state machine (BLOCK statefulness)
<code>sanity_check.py</code>	Five baseline policies + dominance assertion

Also propagated `maintain_order=True` to the Phase 1 `splits.py` / `scalers.py` sort calls (3 duplicate `startOffset` ties found in one intra-file-split file) to guarantee the environment reproduces the exact same row ordering those modules assumed when computing row-range boundaries.

6. Next Steps

This closes Phase 2. Phase 3 trains the RL-only baseline (RecurrentPPO via `sb3-contrib`, base reward, no LLM shaping yet) — the first arm of the eventual 3-way comparison (supervised baselines vs. RL-only vs. RL+LLM).